Chapter 3: State-Space Models

Honours Time Series Module

September 2026

Introduction

State-space models are models for time series data in which we model 1.) a vector of, usually unobserved, state variables which change over time (e.g. level, slope, seasonal effect, true location, true population level) and 2.) conditional on these states, a series of observations. Many common models such as regression and ARIMA models can be written as special cases of state-space models.

The state-space approach has several advantages: stationarity is not required, instead trend and seasonality are modelled explicitly instead of being treated as nuisance parameters. In addition, parameters such as seasonal effects, means, slopes are allowed to change over time. These features allow for a flexible modelling approach of complex, dynamic systems. For this reason, if we are also assuming a Gaussian process, these models are often called *dynamic linear models*, 'dynamic' because the state variables are allowed to change over time. State-space models can deal with missing values, something which ARIMA models are not good at. The Kalman Filter (a recursive algorithm to predict and update forecasts) is applicable for Gaussian linear state-space models. State-space models are also very useful if we don't observe the variable of interest, but only some noisy version of this (e.g. Argos satellite data on animal trajectories, a situation where we cannot ignore measurement error). In state-space models, the state is assumed to follow an autoregressive process. This simplifies forecasting compared to ARMA models, because we only need to keep track of the last few observations and estimates.

The Kalman Filter was originally developed in the area of space travel, where the true position and speed of a space rocket or satellite are unknown (but of primary interest). One can obtain measurements on the distance and angles, which will depend on the state variables. The aim then is to constantly update estimates of the state variables, given the observations (Meinhold and Singpurwalla, 1983).

Other examples where a state-space model is reasonable can be found in ecology or medicine where we may have an underlying true population level, and conditional on this can model observed counts of viruses or animals. The state variable of interest here would be the true population level and how this changes over time.

Examples in economics could include true demand level and a realisation, true unemployment rate, true regression coefficients.

We can also think about a state-space model as a form of

OBSERVATION = SIGNAL + NOISE

Here we assume that the signal is a linear combination of variables called **state variables** (which define the state of the system at time t).

The State-Space Model

The general state-space model has two equations: one for the dynamics of the state variables (state equation) and one for the observations (observation or measurement equation).

We will denote the vector of state variables as θ. θ_t is not directly observable.

A simple model for the observations is

$y_t = \theta_t + e_t, \qquad e_t \sim iidN(0, \sigma_e^2)$

which could just represent a mean (changing over time) plus random noise.

y_t and θ_t may both be vectors. We want to use the observations y_t to make inference about θ_t.

If θ_t is an $m \times 1$ state vector we can write this more generally as

$$\boldsymbol{Y_t = F_t\theta_t + v_t, \qquad v_t \sim N(0, V)}$$

The observation (vector) is a linear function of state variables. $\boldsymbol{F}_t$ is assumed known, and we will assume that it does not change over time. This means that for the purpose of these notes we will drop the subscript from $\boldsymbol{F}$.

The above is called the **observation or measurement equation**.

The **state equation** models the dynamics of the state variables:

$$\boldsymbol{\theta_t = G_t\theta_{t-1} + w_t, \qquad w_t \sim N(0, W)}$$

$\boldsymbol{G}_t$ is $m \times m$, assumed known, and again, in these notes we will drop the subscript, implying that G does not change over time. $\boldsymbol{\theta}_t$ is modelled as a Markov chain. The equation describing the dynamics of θ is called the **state**, **system** or **transition equation**.

If Y_t is univariate, then $W = \sigma_w^2$, $V = \sigma_v^2$.

The errors in the observation and state equations are assumed to be serially uncorrelated and uncorrelated with each other. Explanatory variables can be added, usually to the right-hand side of the observation equation.

Sometimes you will see F_t', sometimes without the transpose. F without transpose is a row vector. `dlm()` uses F without transpose.

Dynamic Linear Model

Linear, Gaussian state-space models are often called *dynamic linear models*. Their general form is

$$\begin{aligned} \boldsymbol{Y_t} &= \boldsymbol{F_t\theta_t} + \boldsymbol{v_t}, \qquad \boldsymbol{v_t} \sim \boldsymbol{N(0, V)} \\ \boldsymbol{\theta_t} &= \boldsymbol{G_t\theta_{t-1}} + \boldsymbol{w_t}, \qquad \boldsymbol{w_t} \sim \boldsymbol{N(0, W)} \end{aligned}$$

Where $\boldsymbol{\theta}$ is a vector of state variables, and $\boldsymbol{Y}_t$ are observations. To initialize the system we add a normal prior

$$\boldsymbol{\theta}_0 \sim N(\boldsymbol{m_0}, \boldsymbol{C_0}).$$

Although the Kalman Filter can be used to set up the likelihood and the parameters can then be estimated by maximum likelihood, the Kalman Filter itself takes a very Bayesian approach, and therefore we also introduce the initial values as priors here.

State Space Models: Extensions

There is a lot more to state-space models, most of which we will not cover:

- multivariate times series (Y_t is a vector)
- non-linear, non-Gaussian state-space models (general state-space models)

  Examples include:

  - exponential family state space models with non-Gaussian observation process.

  - Hidden Markov models (special kind of state-space model with discrete states), see next chapter

  - stochastic volatility models: models for changing variance

- adding explanatory variables

  $\theta_t = G\theta_{t-1} + \beta x_t + w_t$

  $y_t = F\theta_t + \gamma z_t + v_t$

In a picture

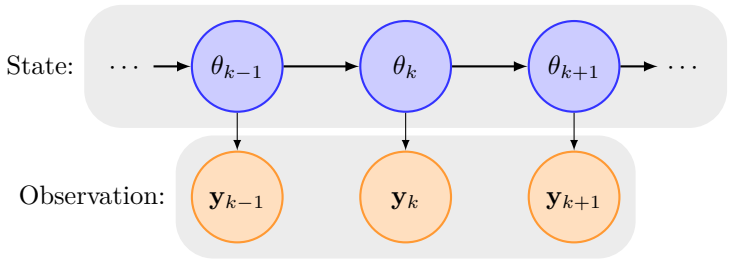


Figure 1: Diagram of a state-space model: state process in blue, observation process in orange

Figure 1 illustrates the dependence structure of a simple state space model. This is a DAG (directed acyclic graph). θ_t is a Markov chain. Conditioning on θ_t the y_t's are independent, y_t depends on θ_t only. The system is completely specified by the initial distribution $\pi(\theta_0)$ and the conditional densities $\pi(\theta_t|\theta_{t-1})$ and $\pi(y_t|\theta_t)$, which means that we can easily construct a likelihood for this system.

Linear Regression Model

A linear regression model can be written in state-space form.

$$Y_t = \theta_1 + \theta_2 x_t + e_t, \qquad e_t \sim iidN(0, \sigma^2)$$

In this model the process variance (in the state equation) is zero, i.e. the state variables θ_1 and θ_2 do not change over time: $\boldsymbol{\theta_t = \theta_{t-1}}$ is not written explicitly here. The state-space model gives us a way to extend this regression model, e.g. we could make the state variables dynamic and would then have a time-varying coefficients model, where the regression coefficients (slopes) are allowed to change over time.

AR(2)

To write ARIMA models in state-space form we need to extend the definition of state-space models. At the same time we introduce exogenous (explanatory) variables.

$$\theta_t = G\theta_{t-1} + Bu_t + Hw_t$$

$$y_t = F\theta_t + Au_t + Re_t$$

Here u_t is a vector of explanatory variables at time t and A and B are coefficient matrices. The added matrices in front of the error terms allow for more flexible models. All of these matrices can change over time (subscript t), but this has been omitted here, to keep the models simpler.

The AR(2) model is

$$y_t = \alpha + \phi_1 y_{t-1} + \phi_2 y_{t-2} + e_t.$$

We can define $\theta_t = (y_t, y_{t-1})$.

Then the state equation is

$$\theta_t = \begin{pmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{pmatrix} \theta_{t-1} + \begin{pmatrix} \alpha \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 0 \end{pmatrix} e_t = \begin{pmatrix} \alpha + \phi_1 y_{t-1} + \phi_2 y_{t-2} + e_t \\ y_{t-1} \end{pmatrix} = \begin{pmatrix} y_t \\ y_{t-1} \end{pmatrix}.$$

And the observation process is

$$y_t = F\theta_t + w_t = \begin{pmatrix} 1 & 0 \end{pmatrix} \theta_t + 0.$$

There are different ways in which one can put an AR(2) model into state-space form (see tutorial exercises). A very good reason for writing ARIMA models in state-space form is that, once a model is in state-space form, we can use the Kalman filter for estimation and prediction, and we can handle missing values. R's `arima()` function uses a state-space representation of the models and the Kalman filter to estimate and predict; same for some (all?) of the exponential smoothing procedures.

ARMA(1, 1)

$$y_t = \mu + \phi_1(y_{t-1} - \mu) + \eta_t + \lambda\eta_{t-1}$$

$$\theta_t = (y_t - \mu, \lambda\eta_t)$$

$$\theta_t = G\theta_{t-1} + H\eta_t = \begin{pmatrix} \phi & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} y_{t-1} - \mu \\ \lambda\eta_{t-1} \end{pmatrix} + \begin{pmatrix} 1 \\ \lambda \end{pmatrix} \eta_t$$

$$y_t = F\theta_t + \mu = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} y_t - \mu \\ \lambda\eta_t \end{pmatrix} + \mu$$

In the last two examples (AR(2) and ARMA(1, 1)) we have put observations into the state vector. This means that the states in this case are observed, and some of the variances are equal to zero.

Some Special State-Space Models

Random Walk plus Noise (= Local Level Model)

$$\begin{aligned} \boldsymbol{Y_t} &= \boldsymbol{\mu_t} + \boldsymbol{v_t}, & \boldsymbol{v_t} &\sim N(\boldsymbol{0}, \boldsymbol{V}) \\ \mu_t &= \mu_{t-1} + w_t, & w_t &\sim N(0, W) \end{aligned}$$

The parameters that need to be estimated are those in V and W, if y_t and θ_t are both univariate, these will be σ_v^2 and σ_w^2.

What are F, G? Is Y_t stationary?

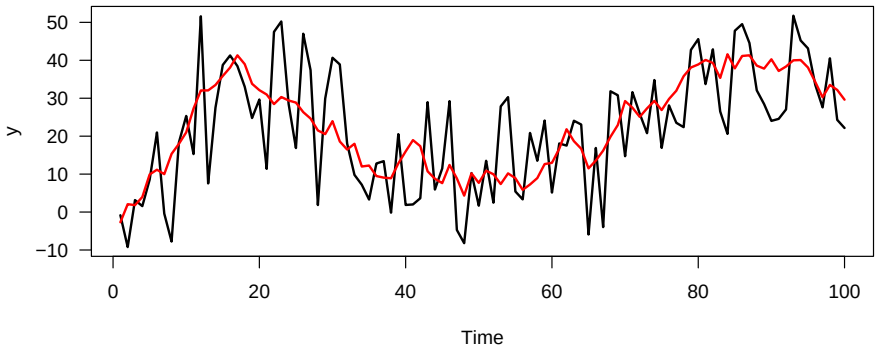


Figure 2: Data simulated from a local level (random walk plus noise) state-space model, with true state (red), and observations (black), observation standard deviation = 10, state standard deviaton = 3.

Linear Growth Model

$$\begin{aligned} Y_t &= \mu_t + v_t, & v_t &\sim N(0, V) \\ \mu_t &= \mu_{t-1} + \beta_{t-1} + w_t, & w_t &\sim N(0, \sigma_w^2) \\ \beta_t &= \beta_{t-1} + \gamma_t, & \gamma_t &\sim N(0, \sigma_\gamma^2) \end{aligned}$$

μ_t = local level, β_t = local trend, which is the current change in level. The level is modelled as a random walk plus drift. This model is also known as **Local Linear Trend**.

What are θ, G, W, F? The three matrices are time-invariant.

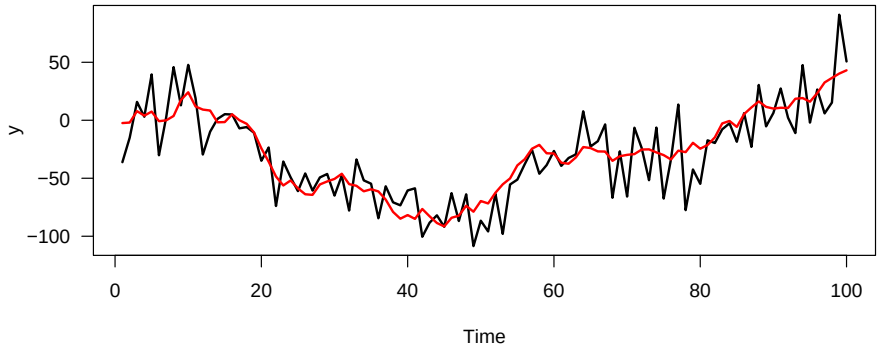


Figure 3: Data simulated from a linear growth state-space model, with true state (red), and observations (black), observation SD = 20, level SD = 5, slope SD = 1.

Basic Structural Model

$$
\begin{aligned}
Y_t &= \mu_t + s_t + v_t, & v_t &\sim N(0, V) \\
\mu_t &= \mu_{t-1} + \beta_{t-1} + w_t, & w_t &\sim N(0, \sigma_w^2) \\
\beta_t &= \beta_{t-1} + \gamma_t, & \gamma_t &\sim N(0, \sigma_\gamma^2)
\end{aligned}
$$

$$
s_t = -\sum_{j=1}^{S-1} s_{t-j} + \delta_t, \qquad \delta_t \sim N(0, \sigma_\delta^2)
$$

s_t = seasonal effect, S = number of observations per year (4 for quarterly, 12 for monthly). The seasonal effects over one year sum to 0: $E(\sum_{j=1}^{S} s_{t-j}) = 0$

What are θ_t, G, W, F? If we have quarterly data

$$
\theta_t = \begin{pmatrix} \mu_t \\ \beta_t \\ s_t \\ s_{t-1} \\ s_{t-2} \end{pmatrix} \qquad F = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \end{pmatrix}
$$

$$
G = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & -1 & -1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}
$$

$$
W = diag(\sigma_w^2, \sigma_\gamma^2, \sigma_\delta^2, 0, 0) = \begin{pmatrix} \sigma_w^2 & 0 & 0 & 0 & 0 \\ 0 & \sigma_\gamma^2 & 0 & 0 & 0 \\ 0 & 0 & \sigma_\delta^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}
$$

The local linear model, linear growth model and basic structural model fall under structural time series models. These are models in which the time series is decomposed (additively) into trend (level), seasonality and irregular variation. They are written in state-space form (see references to Harvey).

Example: Nile

The Nile data set contains annual flow of the river Nile at Aswan, 1871-1970, in 100 million cubic metres. There appears to be a sudden drop in 1898. Construction on the Aswan low dam started in 1898 (R `datasets`).

```
library(dlm)
plot(Nile, las = 1)

mod.ll <- dlmModPoly(order = 1, dV = 15100, dW = 1468)
filt <- dlmFilter(Nile, mod.ll)
lines(dropFirst(filt$m), col = "blue", lwd = 2)
```

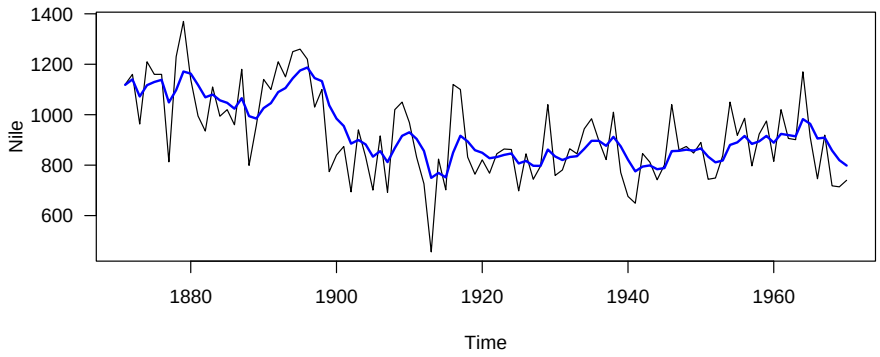


Figure 4: Nile data with filtered level (blue).

In this example, modelling a 'true' underlying level is more an approach to understand dynamics from year to year. `dV` and `dW` refer to the diagonal elements of the observation and state variance matrices, respectively. Here, the variances are assumed known for illustration purposes. Later we will see how these can be estimated. The blue line represents *filtered* values, which are state estimates at time t, having observed y_t (similar to ℓ_t in exponential smoothing.)

`dlmModPoly(order = 1)` defines a *local level model*.

`statespacer` is a more recent R package which seems to be easier to use. For now, I will leave code for both packages (`dlm` and `statespacer`) in these notes. Both packages allow changing matrices for more flexible models.

```
library(statespacer)

y <- matrix(Nile)

ssr.fit <- statespacer(y = y,
                       local_level_ind = TRUE,
                       initial = 0.5*log(var(y)))

se.smooth <- sqrt(unlist(ssr.fit$smoothed$V[1,1,]))

plot(Nile, las = 1, ylim = c(300, 1400))
lines(1871:1970, ssr.fit$smoothed$level, col = cols[1], lwd = 2)
lines(1871:1970, ssr.fit$smoothed$level + qnorm(0.95) * se.smooth,
      col = 'gray')
lines(1871:1970, ssr.fit$smoothed$level - qnorm(0.95) * se.smooth,
      col = 'gray')
lines(1871:1970, ssr.fit$filtered$level, col = cols[2], lwd = 2)
lines(1871:1970, ssr.fit$predicted$yfit, col = cols[3], lwd = 2)
legend("bottomright", col = cols[1:3], lty = 1, bty = "n", lwd = 2,
       legend = c("smoothed level", "filtered level", "1-step predictions"))
```

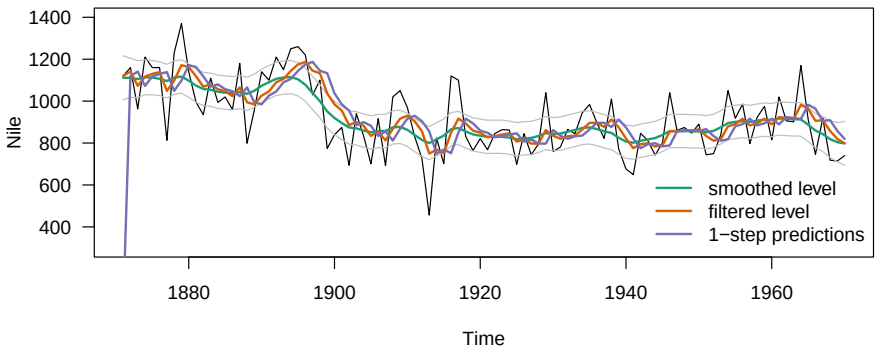


Figure 5: Nile data (black) with smoothed level (blue) and 90% confidence intervals.

Example: Global Temperature Deviations

In this example a linear growth model is fitted to global mean land temperature deviations (from 1951-1980 average), measured in degrees centigrade, for the years 1880-2017, averaged over the Earth's land area (Shumway & Stoffer Chapter 6 Example 6.7).

$$\theta_t = \theta_{t-1} + \delta + w_t$$
$$y_t = \theta_t + v_t$$

Constant slope (change in level) δ. This is implemented by setting the variance for the slope equation to zero (2nd element of W).

```
library(astsa)
plot(gtemp_land, las = 1)

mod.lg <- dlmModPoly(dV = 0.085^2, dW = c(0.033^2, 0))
```

```
filt <- dlmFilter(gtemp_land, mod.lg)
lines(dropFirst(filt$m[,1]), col = "blue", lwd = 2)
```

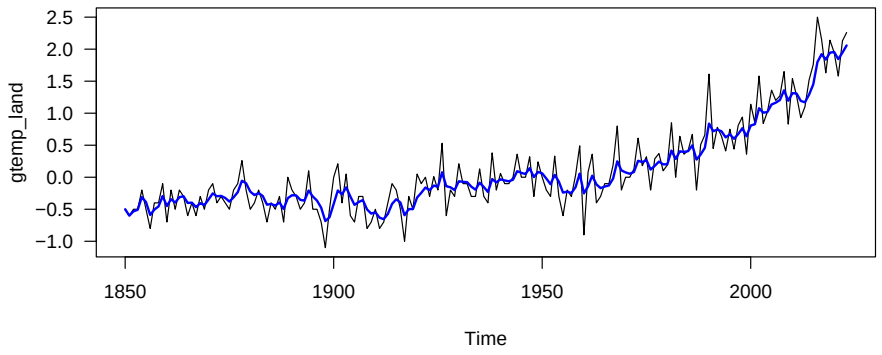


Figure 6: Global temperature data with filtered level (blue).

The default of `dlmModPoly` is to fit a model of order 2, which is the local linear trend model (= linear growth model). Again the variances are assumed known.

`model$m` contains the filtered values. These start one time step before the first observation; hence, we drop the first value for plotting.

```
library(statespacer)
library(astsa)

y <- matrix(gtemp_land)
years <- c(time(gtemp_land))

ssr.fit <- statespacer(y = y,
                   #local_level_ind = TRUE,
                   slope_ind = TRUE,
                   initial = 0.5*log(var(y)))

se.smooth <- sqrt(unlist(ssr.fit$smoothed$V[1,1,]))

plot(years, y, las = 1, type = "l") #, ylim = c(300, 1400))
lines(years, c(ssr.fit$smoothed$level), col = cols[1], lwd = 2)
lines(years, ssr.fit$smoothed$level + qnorm(0.95) * se.smooth,
      col = 'gray')
lines(years, ssr.fit$smoothed$level - qnorm(0.95) * se.smooth,
      col = 'gray')
lines(years, ssr.fit$filtered$level, col = cols[2], lwd = 2)
lines(years, ssr.fit$predicted$yfit, col = cols[3], lwd = 2)
legend("topleft", col = cols[1:3], lty = 1, bty = "n", lwd = 2,
       legend = c("smoothed level", "filtered level", "1-step predictions"))
```

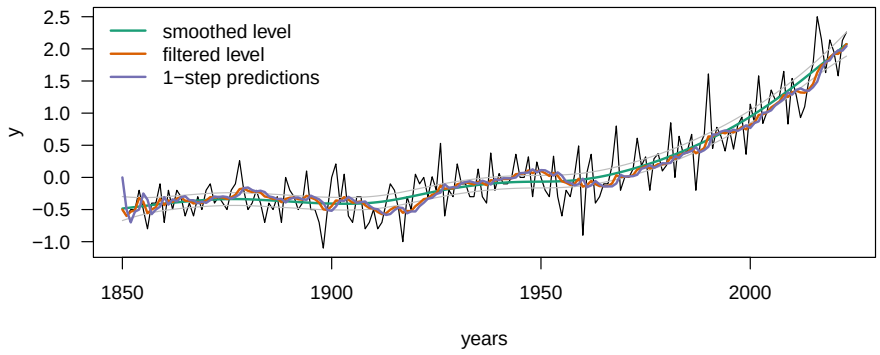


Figure 7: Global temperature deviations, with smoothed and filtered level, and 1-step ahead predictions.

Example: Cape Town Temperatures

In this third example Cape Town mean monthly temperatures are modelled using a basic structural model (with a few adaptations). Data source: https://www.kaggle.com/datasets/sudalairajkumar/daily-temperature-of-major-cities/.

```
library(dplyr)
dat <- read.csv("city_temperature.csv", header = TRUE)
cpt <- filter(dat, City == "Capetown")
cpt <- select(cpt, Month, Day, Year, AvgTemperature)
```

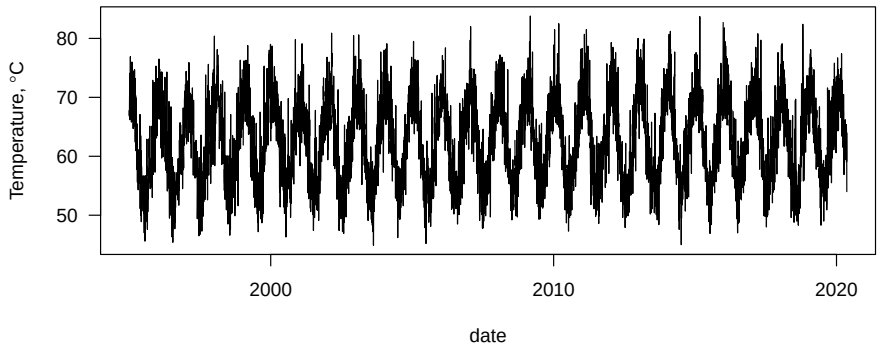


Figure 8: Cape Town average daily temperature since 1995.

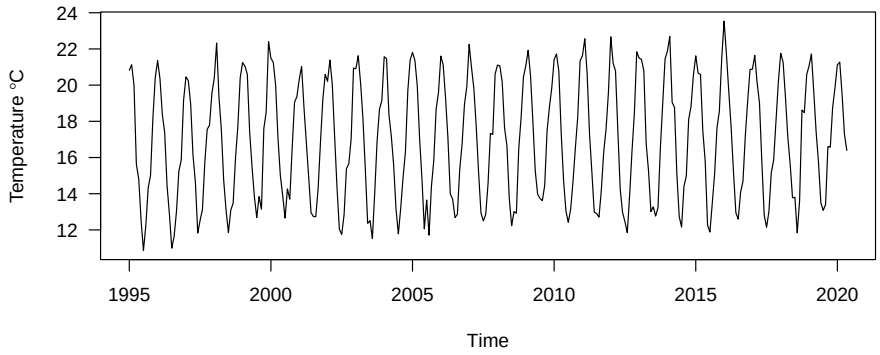


Figure 9: Cape Town average monthly temperature since 1995.

Seasonal effects (`dlmModSeas`) are added to a local level model. The variance in the seasonal component has been set equal to zero, resulting in constant seasonal effects (this is not necessarily the best model).

```
library(dlm)
bsm <- dlmModSeas(frequency = 12, dV =  0.47, dW = rep(0, times = 11)) +
  dlmModPoly(1, dV = 0, dW = 0.0026)
filt <- dlmFilter(mt, bsm)
plot(mt, las = 1, ylab = expression(paste("Temperature ",degree,"C")))
lines(dropFirst(filt$m[,12]), col = "blue", lwd = 2)
```

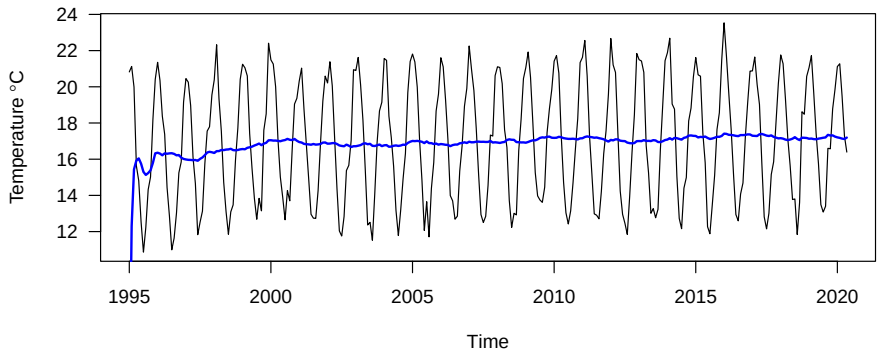


Figure 10: Cape Town mean monthly temperature since 1995 with fitted level (blue).

```
plot(dropFirst(filt$m[,1]), col = "blue", lwd = 2,
     ylab = "seasonal effects", las = 1)
```

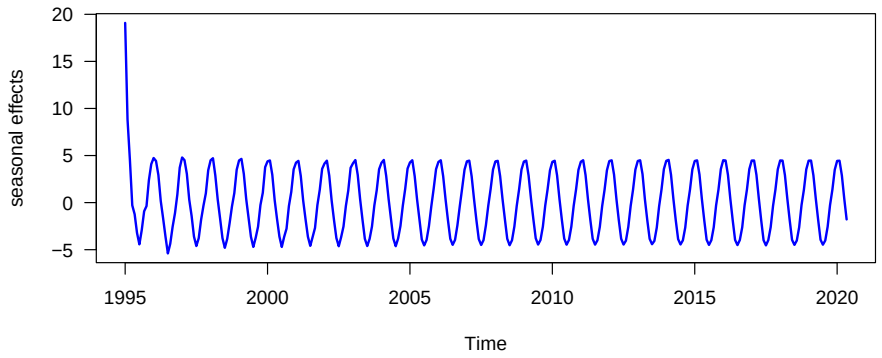


Figure 11: Cape Town mean monthly temperature since 1995 with estimated (filtered) seasonal effects (blue).

```
library(statespacer)

y <- matrix(mt)
times <- c(time(mt))

ssr.fit <- statespacer(y = y,
                       local_level_ind = TRUE,
                       BSM_vec = 12,  # monthly
                       format_BSM_list = list(matrix(0)),
                       initial = log(var(y)))

se.smooth <- sqrt(unlist(ssr.fit$smoothed$V[1,1,]))

plot(times, y, las = 1, type = "l", ylim = c(5, 25),
     xlab = "")
```

```
lines(times, ssr.fit$smoothed$level, col = cols[1], lwd = 2)
lines(times, ssr.fit$smoothed$level + qnorm(0.95) * se.smooth,
      col = 'gray')
lines(times, ssr.fit$smoothed$level - qnorm(0.95) * se.smooth,
      col = 'gray')
lines(times, ssr.fit$filtered$level, col = cols[2], lwd = 2)
lines(times, ssr.fit$predicted$yfit, col = cols[3], lwd = 2)
legend("bottomright", col = cols[1:3], lty = 1, bty = "n", lwd = 2,
       legend = c("smoothed level", "filtered level", "1-step predictions"))
```

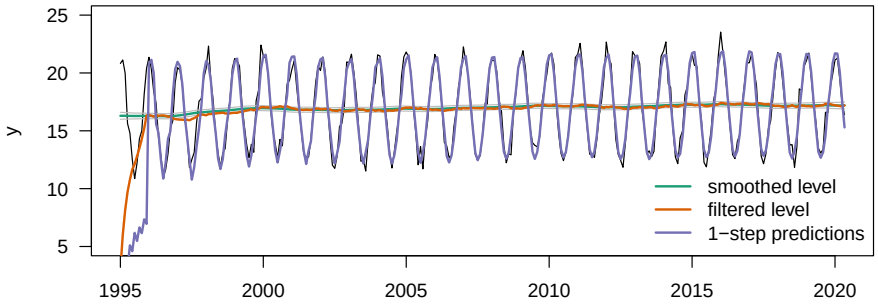


Figure 12: Global temperature deviations, with smoothed and filtered level, and 1-step ahead predictions.

In these last two plots the initial change in the blue line is due the choice of starting value (prior) for the initial state.

```
res <- residuals(filt)$res
par(mfrow = c(1, 4))

plot(res, las = 1)
acf(res, main = "", xlim = c(1, 2))
pacf(res, main = "", ylim = c(-0.2, 1))
hist(res, main = "", las = 1, xlab = "residual")
```

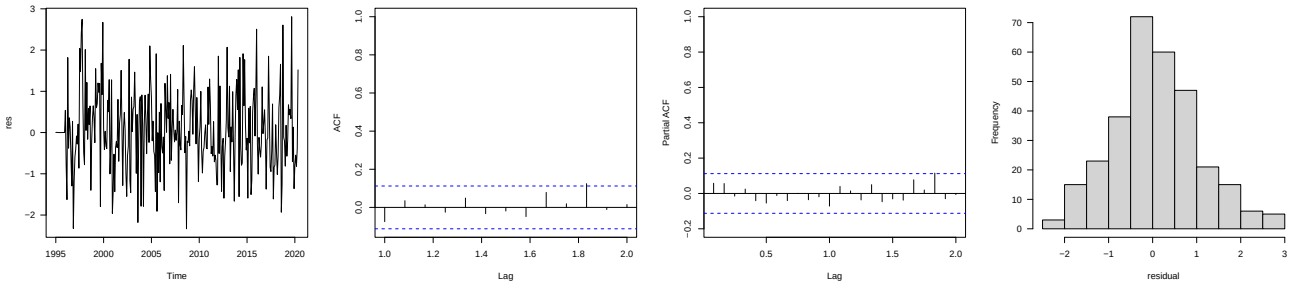


Figure 13: Residual plots from basic structural model fitted to Cape Town mean monthly temperature data. Lag = 1 equals one year.

Instead of using 12 seasonal effects we could use Fourier / trigonometric terms. Here we specify $s = 12$ as the number of observations per full cycle, and $q = 2$ the number of sine and cosine pairs (harmonic

terms) to add. The `dW = 0` argument means that we are modelling the seasonal effect as constant (not allowed to change over time).

```
par(mfrow = c(1, 2))
sm2 <- dlmModPoly(1, dV = 0.47, dW = 0.0025) +
  dlmModTrig(s = 12, q = 2, dV =  0.47, dW = 0)

sm2.filt <- dlmFilter(mt, sm2)
plot(mt, las = 1, ylab = "Monthly mean temperature")
lines(dropFirst(sm2.filt$m[,1]), col = "blue", lwd = 2)
plot(dropFirst(sm2.filt$m[,2]), col = "blue", lwd = 2,
     ylab = "seasonal effect", las = 1)
```

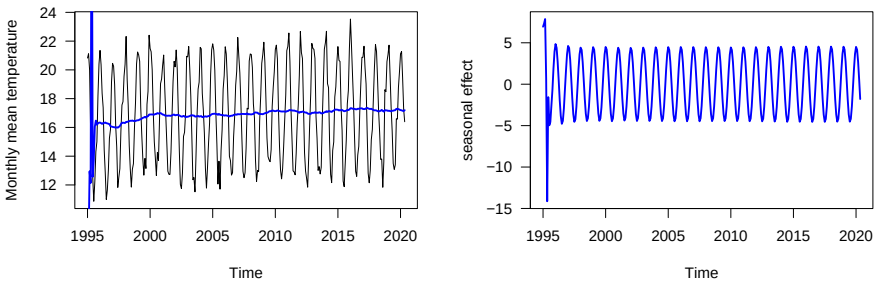


Figure 14: Cape Town mean monthly temperature with filtered level estimates (blue). RHS: seasonal effect using two harmonic pairs. Model fitted: local level plus seasonal effects.

Filtering, Smoothing, Forecasting

The aim of a state space analysis is estimating and forecasting the underlying unobserved signal μ_t, given (noisy) data up to time s. There are three special cases of this:

- $s < t$: forecasting/prediction
- $s = t$: filtering
- $s > t$: smoothing

Forecasting and filtering are done using the **Kalman filter**, smoothing using the **Kalman smoother**. Smoothing refers to obtaining best estimates of previous states, given all of the observed information, the aim is to smooth noisy data. Filtering refers to filtering out noise.

The Kalman Filter: Overview

The Kalman filter is a famous algorithm. It was involved in Apollo's navigation computer, and is still used in satellite navigation systems, cell phones, and many computer games Faragher (2012). It is used to filter and forecast in dynamic linear Gaussian systems (all noise is assumed normal). For such systems the Kalman filter is optimal, in that it minimises the mean squared error.

The Kalman filter is recursive, which allows us to use it for processing new observations as they arrive. It is easy to program and can be used for any model that can be cast into linear Gaussian state-space model form. For non-linear, non-Gaussian problems there are extensions of the Kalman filter, which we will not cover here.

Examples of non-linear and/or non-Gaussian state-space models include dynamic generalized linear models (exponential family state-space models, e.g. for binary or count data), and stochastic volatility models.

The Kalman filter takes a Bayesian approach, in that all estimates of current states and uncertainties are expressed as probability distributions. In the dynamic linear model all processes follow a Gaussian distribution. This is partly why the Kalman filter results in a simple algorithm: Gaussian priors and likelihoods result in a Gaussian posterior.

posterior $\propto$ likelihood $\times$ prior

$$\pi(\theta_t|y_t) \approx \pi(y_t|\theta_t) \times \pi(\theta_t|y_{t-1})$$

In what follows below I will keep using green for the prior distribution and purple for the posterior, black for the likelihood.

The Kalman Filter

We start off by assuming that we have updated estimates of the state variables for time $t-1$ given observations up to and including time $t-1$:

$$\theta_{t-1}|y_{1:t-1} \sim N(m_{t-1}, C_{t-1}) \tag{1}$$

This will start off with the initial distribution for θ_0: $\theta_0 \sim N(m_0, C_0)$. Usually m_0 is set equal to zero, and C_0 to a very large value.

One-step-ahead predictive distribution for $\theta_t|y_{1:t-1} : N(a_t, R_t)$

$$a_t = Gm_{t-1}, \qquad R_t = GC_{t-1}G' + W$$

This follows from the process equation.

The one-step-ahead predictive distribution for $Y_t|y_{1:t-1} : N(f_t, Q_t)$

$$f_t = Fa_t, \qquad Q_t = FR_tF' + V \tag{2}$$

This follows from the observation equation. In the next step we calculate the filtering distribution for $\theta_t|y_{1:t} : N(m_t, C_t)$

$$m_t = a_t + R_tF'Q_t^{-1}e_t, \qquad C_t = R_t - R_tF'Q_t^{-1}FR_t \tag{3}$$

Sate-space model:

$Y_t = F_t\theta_t + v_t, \qquad v_t \sim N(0, V)$

$\theta_t = G_t\theta_{t-1} + w_t, \qquad w_t \sim N(0, W)$

If $X \sim N(\mu, \Sigma)$, then

$CX \sim N(C\mu, C\Sigma C')$. Errors and state process are uncorrelated.

Table 1: Summary of Kalman filter prediction and update equations.

	$t=0$		$t=1$		$t=2$	
$\hat{\theta}_t \vert y_{t-1}$			a_1, R_1		a_2, R_2	prior
		$\nearrow$	$\downarrow$	$\nearrow$	$\downarrow$	
$\hat{\theta}_t \vert y_t$	m_0, C_0		m_1, C_1		m_2, C_2	posterior
			$\uparrow$		$\uparrow$	
y_t			y_1		y_2	likelihood

where $e_t = y_t - f_t$ (Petris et al. (2009)). More details for deriving this filtering distribution will follow below.

In summary, given an estimate for θ based on the most current observation, we can use this to make a forecast / prediction for the next time step. Then, once the observation for this time step comes in we update our prediction and obtain a posterior, updated estimate of the state, and so on. This process is visually illustrated in Figure 15, and in Table Table 1.

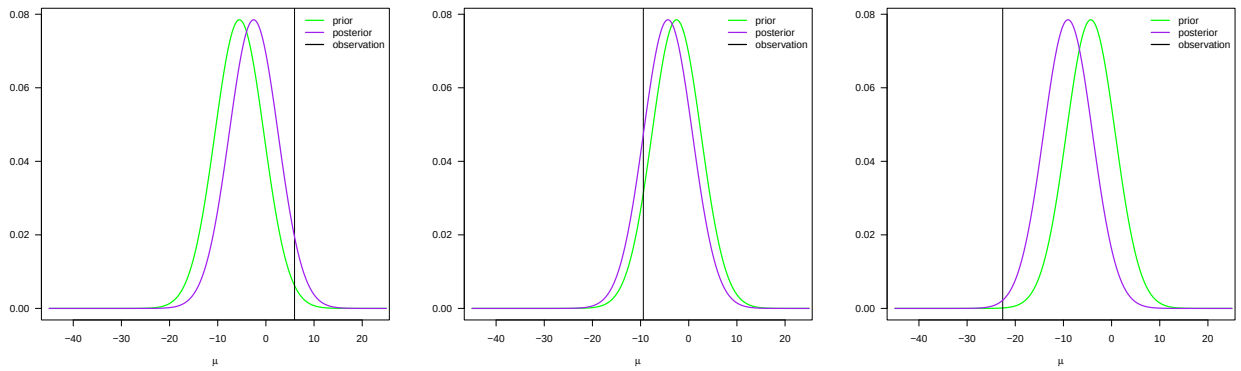


Figure 15: Three iterations of the Kalman filter, prior in green, vertical black line = observation, posterior in purple; posterior becomes prior in next step (almost). The posterior is a compromise between prior and observation.

Kalman Filter Derivation

This section is based on Meinhold and Singpurwalla (1983). The last step in deriving the Kalman Filter is to find the posterior distribution $[\theta_t|Y_t]$. For this we

1. Rewrite the likelihood $[Y_t|\theta_t, Y_{t-1}]$ in terms of the error $[e_t|\theta_t, Y_{t-1}]$.
2. Find the joint distribution $[e_t, \theta_t|Y_{t-1}]$.

$$[e_t, \theta_t|Y_{t-1}] = [e_t|\theta_t, Y_{t-1}] \times [\theta_t|Y_{t-1}]$$

3. Find the posterior $[\theta_t|e_t, Y_{t-1}]$, using a standard result for conditional multivariate normal distributions.

STEP 1.

$$\begin{aligned} e_t &= Y_t - \hat{Y}_t \\ &= Y_t - FG\hat{\theta}_{t-1} \\ &= (F\theta_t + v_t) - FG\hat{\theta}_{t-1} \\ &= F(\theta_t - G\hat{\theta}_{t-1}) + v_t. \end{aligned}$$

From this follows that $E[e_t|\theta_t, Y_{t-1}] = F(\theta_t - G\hat{\theta}_{t-1})$ and $Var[e_t|\theta_t, Y_{t-1}] = V$.

Y_t is the observation, once we have observed Y_t, we have observed e_t, as no extra information is required (besides the conditional information). Therefore we can use e_t instead of Y_t in the likelihood.

STEP 2.

The converse of the following result from multivariate normal theory will be used for this step.

If

$$\begin{pmatrix} X_1 \\ X2 \end{pmatrix} \sim N\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix}\right)$$

then

$$X_1|X_2 \sim N(\mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2), \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}).$$

We have $[e_t|\theta_t, Y_{t-1}]$ from step 1, and $\theta_t|Y_{t-1}$ from eq. 2. This information is combined to obtain the joint probability density function for e_t and θ_t. Then, again using the above, we can obtain the posterior distribution $[\theta_t|e_t, Y_{t-1}]$.

If we define $X_1 = e_t|Y_{t-1}$, and $X_2 = \theta_t|Y_{t-1}$, then

- $\mu_2 = G\hat{\theta}_{t-1}$
- $\Sigma_{22} = R$
- $\mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2) = F(\theta_t - G\hat{\theta}_{t-1})$.

  Therefore,

- $\mu_1 = 0$
- $\Sigma_1 1 = V + FRF'$
- $F = \Sigma_{12}\Sigma_{22}$ so that $\Sigma_{12} = RF'$

- $\Sigma_{21} = FR$

$$\begin{pmatrix} \theta_t \\ e_t \end{pmatrix} \sim N\left(\begin{pmatrix} G\hat{\theta}_{t-1} \end{pmatrix}, \begin{pmatrix} R & RF' \\ FR & V + F'RF \end{pmatrix} \right)$$

STEP 3.

$$[\theta_t | e_t, Y_{t-1}] \sim N\left(G\hat{\theta}_{t-1} + RF'(V + FRF')^{-1} e_t, R - RF'(V + FRF')^{-1} FR \right)$$

$G\hat{\theta}_{t-1}$ was the mean of the prior. Therefore the above can be seen as an updating equation depending on the observed error.

$K_t = R_t F'(V_+ F R_t F')^{-1} = R_t F' Q_t^{-1}$ is also called the *Kalman gain matrix*, referring to the information obtained from the observation.

The Kalman filter can be used for any system that can be written into linear Gaussian state-space form.

If V and R are known this is a recursive algorithm starting from $\theta_0 \sim N(m_0, C_0)$ and proceeding through the forecasting and updating steps, recursively.

If there are parameters which are unknown, the Kalman filter is used to set up the likelihood function, and parameters can be estimated using maximum likelihood, or a Bayesian approach.

Example

$$Y_t = \theta_t + v_t$$

$$\theta_t = \theta_{t-1} + w_t$$

This is the local level model (Meinhold and Singpurwalla (1983) call this the steady model).

Here, $F = G = 1$, and we assume that $\Sigma_0 = C_0 = 1, V = 2, W = 1$ are known.

```
## simulate data
set.seed(21)
N <- 110
W = 1
V = 2

theta <- numeric(N)
y <- numeric(N)
```

```
theta[1] <- 0

for (i in 2:N) {
 theta[i] <- rnorm(1, theta[i-1], sqrt(W))
 y[i] <- rnorm(1, theta[i], sqrt(V))
}

theta <- theta[-c(1:10)]
y <- y[-c(1:10)]

plot.ts(y)
lines(theta, col = "red")

N <- 100
```

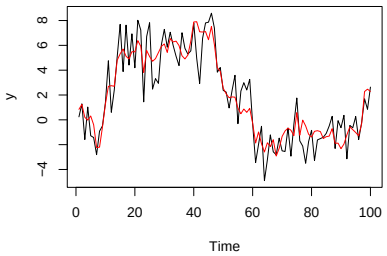


Figure 16: Data (black) simulated from a local level model, with true state in red.

```
KF <- function(FF, GG, m0, C0, W, V, y) {
  N <- length(y)
  at <- numeric(N + 1)
  Rt <- numeric(N + 1)
  ft <- numeric(N)
  Qt <- numeric(N)
  mt <- numeric(N)
  Ct <- numeric(N)
  et <- numeric(N)

  ## predict theta
  at[1] <- m0
  Rt[1] <- GG %*% C0 %*% GG + W

  for (i in 1:N) {

    ## predict y
    ft[i] = FF %*% at[i]
    Qt[i] = FF %*% Rt[i] %*% FF + V

    ## posterior
    if (is.na(y[i])) {
      et[i] = 0
      Kt = 0
    } else {
      Kt = Rt[i] %*% FF %*% solve(Qt[i])
      et[i] <- y[i] - ft[i]
    }

    mt[i] = at[i] + Kt %*% et[i]
    Ct[i] = Rt[i] - Kt %*% FF %*% Rt[i]
```

```
    ## predict theta
    at[i+1] <- GG %*% mt[i]
    Rt[i+1] <- GG %*% Ct[i] %*% GG + W
  }

  return(list(at = at, Rt = Rt, ft = ft, Qt = Qt, mt = mt, Ct = Ct))
}
```

```
## initial distribution
m0 <- 0
C0 <- 1

GG = 1
FF = 1

y1 <- c(y, rep(NA, times = 5))
y.kf <- KF(FF = 1, GG = 1, m0 = m0, C0 = C0, W = 1, V = 1, y1)

all <- cbind(y1,
             theta,
             y.kf$mt,
             y.kf$mt - 2 * sqrt(y.kf$Ct),
             y.kf$mt + 2 * sqrt(y.kf$Ct))

plot.ts(all, plot.type = "single", col = c("black", "red", "blue", "grey", "grey"),
        lwd = 2, las = 1, ylab = "y")
legend("topright", lwd = 2, lty = 1, col = c("black", "red", "blue", "grey"),
       legend = c("observed", "true", "filtered", "95% confidence limits"), bty = "n")
```

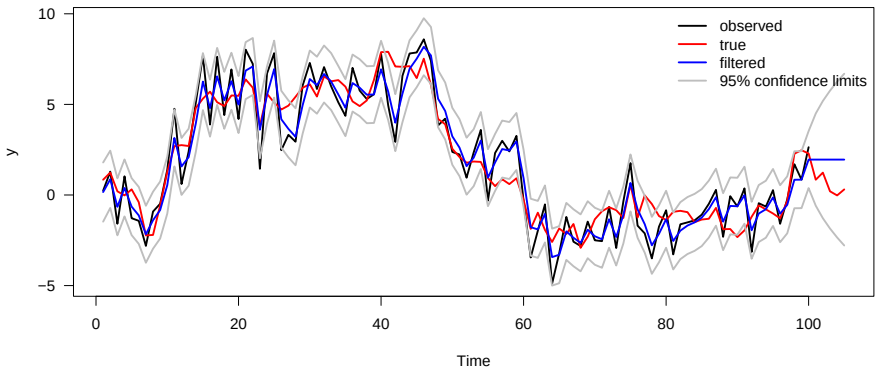


Figure 17: Filtered values for local level model estimated using Kalman filter (blue), observations (black), true state (red), 95% confidence interval calculated as using 2 standard errors (grey).

The confidence intervals in **?@fig-kalman-ll** were calculated from $[\theta_t|y_t, \theta_{t-1}]$, filtered state values.

Prediction Intervals

We can obtain prediction intervals from the one-step-ahead predictive distribution for $[y_t|y_{t-1}] \sim N(f_t, Q_t)$.

We also use the *in-sample* one-step forecasts to compare and check our models, and for calculating residuals.

For forecasts into the future, y_{T+h} is not available. This means that in the update step, there is no new information: the Kalman gain equals 0 (the state information is not updated with any new observation obtained from an observation). This leads to:

$$m_t = a_t$$

$$C_t = R_t$$

Note that we can treat missing observations in exactly the same way (see next section). I have implemented forecasting in the above Kalman Filter function `KF` by attaching a few missing values to the observation vector `y`.

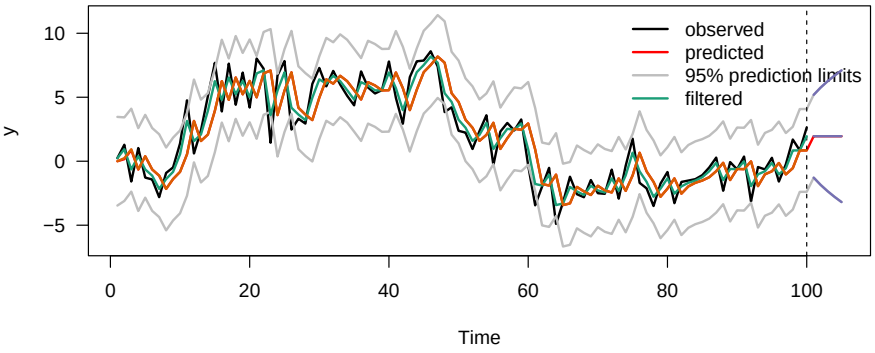


Figure 18: Simulated data from local level model (black) with predicted (one step-ahead predictions (red)) and prediction intervals.

As a check, forecasts from `dlm` result in exactly the same forecasts and prediction intervals.

Likelihood

Likelihood is defined as the probability of the observed data as a function of parameters ϕ. We can break up the joint probability distribution into a product of conditional distributions, and therefore can write the likelihood as:

$L(\phi) = p(y_1, ...y_n; \phi) = \prod_{t=1}^{n} p(y_t|y_{1:t-1}; \phi)$

From the Kalman filter derivations we have

$$p(y_t|y_{1:t-1}; \phi) \sim N(f_t, Q_t)$$

Chain rule: $P(A_1 \cap A_2 \cap ...A_n) = P(A_n|A_1 \cap ...A_{n-1})P(A_1 \cap ...A_{n-1})$. Continuing in this way we can write the joint pdf as a product of conditional distributions.

so that the log-likelihood becomes

$$\ell(\phi) = -\frac{1}{2}\sum_{t=1}^{n}\log|Q_t| - \frac{1}{2}\sum_{t=1}^{n}(y_t - f_t)'Q_t^{-1}(y_t - f_t).$$

This likelihood can be difficult to maximize, with many local maxima, and flat peaks. It is therefore important to check using different starting values Petris et al. (2009).

Missing Observations

The Kalman Filter deals with missing observations in the following way: if y_t is missing, it carries no information and we cannot update our prediction (posterior = prior).

$$\pi(\theta_t|y_{1:t}) = \pi(\theta_t|y_{1:t-1}).$$

I.e., the filtering distribution at time t equals the one-step-ahead predictive distribution at time $t-1$.

$\theta_t|y_{1:t-1} \sim N(a_t, R_t) \rightarrow$ replace $m_t = a_t$ and $C_t = R_t$. (This is equivalent to setting $F_t = 0$ or $V_t = \infty$.)

Either way leads to gain matrix $K_t = R_t F' Q_t^{-1} = 0$.

Uncertainty is added in each step (with missing data) when predicting the state for the following time step, through the process variance.

Example: Nile data with missing values

Here we fit a local level model to the Nile data, then remove some values and examine the change in confidence intervals.

```
nm <- Nile
nm[40:45] <- NA    ## 1910-1915

lik.fun <- function(x) {
 dlmModPoly(1, dV = exp(x[1]), dW = exp(x[2]))
}

# maximize the likelihood
fit2 <- dlmMLE(nm, parm = c(0,0), build = lik.fun)
fit2$conv
fit2
exp(fit2$par)
```

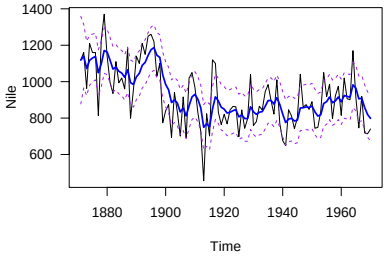


Figure 19: Nile data with filtered level estimates and 95% confidence intervals, from a local level model.

```
nm.model <- lik.fun(fit2$par)
model.filtered2 <- dlmFilter(nm, nm.model)

plot.ts(Nile, las = 1, xlim = c(1900, 1930), lwd = 2, xlab = "")
lines(1871:1970, model.filtered$m[-1], col = cols[1], lwd = 2)

tt <- length(model.filtered$m)
tt

CC <- numeric(tt)

U.C <- model.filtered$U.C
D.C <- model.filtered$D.C

t <- 2
for (t in 1:tt) {
  CC[t] <- U.C[[t]] %*% (D.C[t,]^2) %*% t(U.C[[t]])
}

CC

lines(1871:1970, model.filtered$m[-1] - 1.96 * sqrt(CC[-1]),
      col = cols[1], lty = 2, lwd = 2)
lines(1871:1970, model.filtered$m[-1] + 1.96 * sqrt(CC[-1]),
      col = cols[1], lty = 2, lwd = 2)

CC2 <- numeric(tt)

U.C <- model.filtered2$U.C
D.C <- model.filtered2$D.C

t <- 2
for (t in 1:tt) {
  CC2[t] <- U.C[[t]] %*% (D.C[t,]^2) %*% t(U.C[[t]])
}

CC2
lines(1871:1970, model.filtered2$m[-1], col = cols[2], lwd = 2)
lines(1871:1970, model.filtered2$m[-1] - 1.96 * sqrt(CC2[-1]),
      col = cols[2], lty = 2, lwd = 2)
lines(1871:1970, model.filtered2$m[-1] + 1.96 * sqrt(CC2[-1]),
      col = cols[2], lty = 2, lwd = 2)

legend("topright", col = cols, lwd = 2,  bty = "n",
       legend = c("filtered: original series", "filtered: with missing"))
```

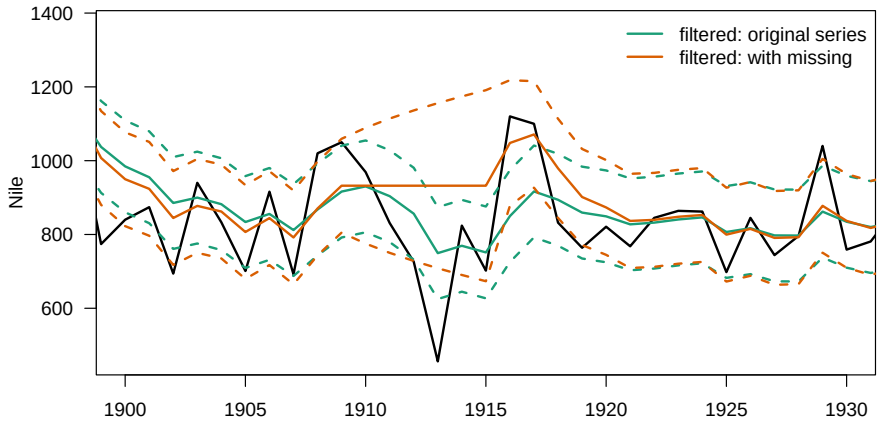


Figure 20: Local level model fitted to Nile data. Filtered state estimates with confidence intervals, without missing values, and with observations 1910-1915 missing.

Kalman Smoother

The equations for the Kalman smoother are presented here, but without proofs. The Kalman smoother starts at the end and works backwards. The *smoothed* state estimates are often used when such estimates are required in hindsight, rather than for predictive purposes.

If $\theta_{t+1}|y_{1:T} \sim N(s_{t+1}, S_{t+1})$ then $\theta_t|y_{1:T} \sim N(s_t, S_t)$ where

$$s_t = m_t + C_t G'_{t+1} R^{-1}_{t+1}(s_{t+1} - a_{t+1})$$

$$S_t = C_t - C_t G'_{t+1} R^{-1}_{t+1}(R_{t+1} - S_{t+1}) R^{-1}_{t+1} G_{t+1} C_t$$

```
NileSmooth <- dlmSmooth(model.filtered)

U.S <- NileSmooth[["U.S"]]
D.S <- NileSmooth[["D.S"]]
hwid <- qnorm(0.025, lower = FALSE) *
        sqrt(unlist(dlmSvd2var(U.S, D.S)))

smooth <- cbind(NileSmooth$s, as.vector(NileSmooth$s) +
                  hwid %o% c(-1, 1))

plot(dropFirst(smooth), plot.type = "s", col = cols[3],
     ylim = range(Nile), lwd = 2, lty = c(1, 2, 2),
     las = 1, ylab = "Nile flow in 100 mill. m^3", xlab = "")

lines(Nile, type = "o")
```

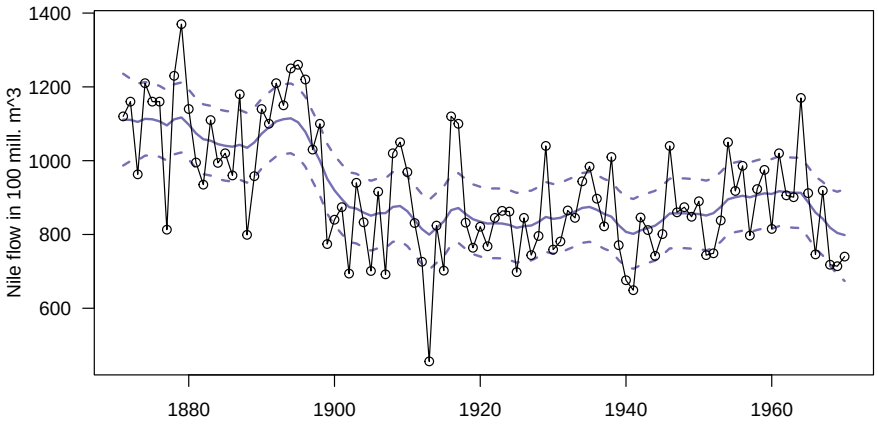


Figure 21: Local level model fitted to Nile data, with smoothed state estimates and 95% confidence intervals.

Bayesian Fitting of State Space Models

The Kalman filter works for Gaussian linear state space models. It gets complicated when state space models are not Gaussian or not linear.

The Bayesian framework is an alternative approach to fitting state space models, including non-linear and non-Gaussian. `stan` is a powerful platform for Bayesian statistical modelling (https://mc-stan.org/). An alternative, similar platform is `JAGS`.

The following `stan` code defines the model, parameters, priors and likelihood, and is saved in a file called `ssm-irw.stan`. All unknowns (including the values of the state variables) are parameters. `stan` automatically chooses sensible priors based on the defined limits for the parameters but it is good practice to define the priors explicitly.

Here we fit an *integrated random walk* structural model to the Nile data. 'Integrated random walk' refers to a local linear trend model with the level process variance set to zero. This means that the *change in level* follows a random walk.

For more information on the Bayesian approach, see the tutorial *Bayesian State Space Models*.

Copy this Stan model code into a file called `ssm-irw.stan`. RStudio knows about Stan files. Leave one line blank at the end of the file.

```
// ssm-irw.stan
// The input data is a vector 'y' of length 'N'.
data {
  int<lower=0> N;
  vector[N] y;
}

parameters {
  real m0;
  real lambda0;
  vector[N] lambda;
```

```
  vector<lower=0>[2] sigma;
}

model {
  vector[N] mu;

  sigma ~ exponential(1);
  m0 ~ normal(0, 1);
  lambda0 ~ normal(0, 1);

  lambda[1] ~ normal(0, sigma[2]);
  mu[1] = m0 + lambda0;

  for (t in 2:N) {
    lambda[t] ~ normal(lambda[t-1], sigma[2]);
    mu[t] = mu[t-1] + lambda[t-1];
  }
  y ~ normal(mu, sigma[1]);
}

generated quantities {
  vector[N] mu;

  mu[1] = m0 + lambda0;
  for (t in 2:N) {
    mu[t] = mu[t-1] + lambda[t-1];
  }
}
```

```
library(cmdstanr)
library(bayesplot)

y <- Nile
N <- length(y)

stan_data <- list(N = N, y = y, N_new = 10)

ssm_irw <- cmdstan_model(stan_file = "./stan/ssm-irw.stan")

fit <- ssm_irw$sample(
  data = stan_data,
  seed = 123,
  chains = 4,
  parallel_chains = 2,
  refresh = 500 # print update every 500 iters
)
```

```
mu_samples <- post |>
              select(starts_with("mu"))
```

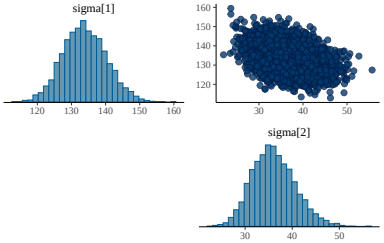


Figure 22: Posterior distribution from an integrated random walk state space model fitted to Nile data. There are two variance parameters.

```
mu_hat <- apply(mu_samples, 2, mean)
mu_ci <- apply(mu_samples, 2, quantile, probs = c(0.025, 0.975))
mu <- t(rbind(mu_hat, mu_ci))

n <- 100
print(dim(mu))
print(length(1871:1980))
plot(y, las = 1, xlab = "", xlim = c(1870, 1980))
matlines(1871:1980, mu, col = "blue", lwd = 1,
        type = "l", lty = c(1, 2, 2))
```

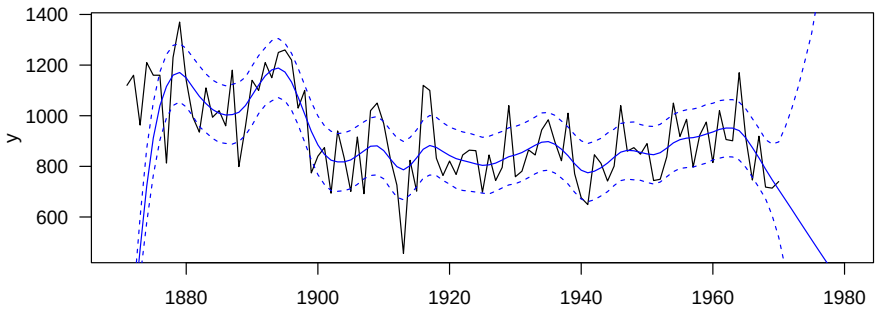


Figure 23: Nile data with estimated level obtained from a Bayesian fit. Model: integrated random walk state space model (level variance equals zero), with 95% credibility interval for the level.

One major advantage of the Bayesian framework is the way in which uncertainty estimates are available directly from the posterior. The estimated level curve from the Bayesian fit corresponds roughly to the smoothed level estimates: state variables are treated as unknown parameters and get estimated. These estimates are much smoother than if we had fitted a local level model.

References

Faragher, Ramsey. 2012. "Understanding the Basis of the Kalman Filter Via a Simple and Intuitive Derivation [Lecture Notes]." *IEEE Signal Processing Magazine* 29 (5): 128–32. https://doi.org/10.1109/MSP.2012.2203621.

Meinhold, Richard J RJ, and Nozer D Singpurwalla. 1983. "Understanding the Kalman Filter." *American Statistician* 37 (2): 123–27. http://www.jstor.org/stable/10.2307/2685871.

Petris, G, S Petrone, and P Campagnoli. 2009. *Dynamic Linear Models with R*. Springer.

- Chatfield. 2004. The Analysis of Time Series. An Introduction. 6th ed. Chapman & Hall. (Chapter 10, State-Space Models and the Kalman Filter)
- Harvey A. 1985. Trends and cycles in macroeconomic time series. Journal of Business & Economic Statistics 3(3):216–27. http://www.jstor.org/stable/10.2307/1391592
- Harvey AC. 1990. Forecasting, Structural Time Series Models and the Kalman Filter. Cambridge University Press.
- Petris G, Petrone S, Campagnoli P. 2009. Dynamic Linear Models with R. Springer.
- Petris G. 2010. An R Package for Dynamic Linear Models. Journal of Statistical Software 36(12):1–16. http://www.jstatsoft.org/v36/i12/paper
- Shumway RH, Stoffer DS. 2018. Time Series Analysis and its Applications. 4th ed. Springer. (Chapter 6, quite theoretical)
- Stan Development Team. 2021. Stan Modeling Language Users Guide and Reference Manual, Version 2.27. https://mc-stan.org